

Безопасность данных (1)

- Проблематика
- Введение
- Фильтрация входных данных
- Фильтрация входных данных с помощью фильтра
- Практика фильтрации данных форм

Проблематика

До сих пор мы отправляли на сервер данные и радовались, что сервер эти данные получает. И для радости были причины, массивы, ассоциативные массивы, инпуты, чекбоксы, радиокнопки, двоичные файлы – все это принималось и обрабатывалось на сервере.

Но жизнь немного сложнее.

В следующем листинге кода HTML-разметка самой обычной формы (form.html), предоставляющей возможность пользователю отсылать на сервер некоторые данные. Серверный файл (server.php) принимает эти данные. Пока все хорошо, протестируйте.

Важно. В примерах я не использую возможности HTML-5 форм. Для полноты эксперимента все поля форм – текстового типа (type=text).

Файл формы.

example_1. form.html. Форма

```
<h1>Безопасность данных</h1>
<h2>Проблематика</h2>
<form action="server.php" method="post">
    Имя: <input type="text" name="name" value="Иван"><p>
```

```
Логин: <input type="text" name="login" value="master"><p>
Е-mail: <input type="text" name="email"
value="john@mail.ru"><p>
Мой сайт: <input type="text" name="site" value="mysuper-
site.ru"><p>
Пароль: <input type="text" name="pwd" value="12345"><p>
<input type="submit" value="Отправить">
</form>
```

Принимающий файл.

example_1. server.php. Проблема отправки данных форм

```
<?php
    if(isset($_POST["name"])){
        // проанализируем принятые данные
        $name = $_POST["name"] ? $_POST["name"] : "No name";
        $login = $_POST["login"] ? $_POST["login"] : "нет
        данных";
        $email = $_POST["email"] ? $_POST["email"] : "нет
        данных";
        $site = $_POST["site"] ? $_POST["site"] : "нет данных";
        $pwd = $_POST["pwd"] ? $_POST["pwd"] : "нет данных";
        echo "<h2>Добрый день, $name!</h2>";
        echo "<h3>Мы получили от вас следующие данные:</h3>";
        echo "Логин: $login <p>";
        echo "Е-mail: $email <p>";
        echo "Ваш сайт: $site <p>";
        echo "Пароль: $pwd <p>";
        echo "<h3>Спасибо за проявленный интерес!</h3>";
```

```
}  
  
?>
```

Все хорошо до определённого момента. А теперь возьмите, и в **любое из полей** формы вставьте следующий код:

```
<script>alert('Привет, мы внедрили на ваш компьютер вредоносную  
программу, и украли данные банковских  
карт...');document.body.insertAdjacentHTML('afterend',  
'<h1>Don't Worry, Be Happy!</h1>');</script>
```

После отправки данных, в HTML разметку принимающего файла server.php будет **внедрен код JavaScript**.

Вывод простой, нельзя доверять пользователю выполнять такое важное действие, как заполнение и отправка форм... Ну, в смысле, **бесконтрольно выполнять**. Все данные пришедшие в вашу систему извне должны быть **проверены и очищены**.

Важно. Вы вполне можете подумать, что ваш первый сайт злоумышленникам не интересен, и проверять данные и писать лишний код совсем не обязательно. На это могу парировать, именно в этот момент, возможно, такой же начинающий, только хакер, открыл книгу "Руководство хакера веб-приложений: поиск и использование недостатков безопасности".

Введение

Безопасность данных является очень важным моментом, который часто недооценивается как разработчиками, так и клиентами.

Зачем очищать и проверять?

В данной теме работаем с информацией, поступающей непосредственно от пользователя, или из других внешних источников. Это означает, что мы не контролируем информацию, которую получаем. Все, что мы можем - это **контролировать то, что будет сделано с этой полученной информацией.**

Практически все виды угроз исходят от информации, передаваемой пользователями или другими третьими лицами.

Основные виды угроз:

- **XSS** (Cross-Site Scripting - межсайтовый скриптинг).

Это способ инъекции кода, когда **скрипт внедряется** в страницу атакуемого вебсайта с совершенно другого сайта на другом сервере.

- **SQL**-инъекция.

Инъекции кода, которая позволяет реализовывать **изменение информации в базе данных** либо иное нарушения нормального функционирования веб-приложения. Осуществляется данная атака путем **внедрения в SQL-запрос** произвольного SQL-кода, предназначенного для взаимодействия с базой данных.

- **CSRF/XSRF** (Cross-Site Request Forgery - подделка межсайтовых запросов).

Обычно такого рода уязвимости возникают при работе с сессиями и cookies и реже - при плохо проверенных и очищенных данных. CSRF может использоваться для выполнения сайтом каких либо запросов без ведома пользователя.

- **Некорректная информация.**

Некорректная информация сама по себе не является "уязвимостью". Однако такая информация во многих случаях приводит к возникновению

ряда проблем, как для владельца сайта, так и для администратора баз данных. Зачастую некорректная по структуре информация приводит к нарушениям в работе, а также к сбоям в работе автоматизированных систем, ожидающих для обработки, **четко структурированные данные в определенном формате.**

Что делать?

Информацию, поступающую из внешних источников, прежде всего, необходимо фильтровать.

Существует два основных типа фильтрации:

- **валидация** (проверка);
- **санитизация** (очистка).

Валидация (проверка) используется для определения соответствия данных определённым критериям. Например, проверить, являются ли введённые данные адресом email, однако, сами данные при этом останутся нетронутыми.

Санитизация (очистка) используется для извлечения из данных нежелательных конструкций. Например, удаление всех символов, которые не должен содержать email-адрес. При этом, проверки данных не происходит.

Фильтрация входных данных

Строго говоря, первый этап валидации начинается на стороне клиента. Этот этап проверки несложно обойти, тем не менее, пренебрегать им не стоит ни в коей мере. Людей рассеянных гораздо больше чем злонамеренных.

Вообще крупные ресурсы валидацию выполняют **несколько раз**:

- на стороне **клиента**;
- на стороне **сервера**;
- на стороне **базы данных**.

Валидация входных данных регулярными выражениями

Перед отправкой данных на сервер важно убедиться, что все обязательные поля формы заполнены данными в корректном формате. Это называется **валидацией** и помогает убедиться, что данные, введённые в каждый элемент формы, соответствуют требованиям.

Валидация данных является неотъемлемой частью работы с формами.

Примечание. Если забыли регулярные выражения, необходимо обратиться к урокам: **Регулярные выражения** курса.

В следующей форме потенциальный пользователь захотел нас удивить и узнал, что такое:

- **теги** (<h1>)
- **скрипты** (<script>)
- **HTML-сущности** (♥ / 😘)

И вставил все это в форму. Вероятно из наилучших побуждений.

example_2. form.html. Форма

```
<form action="server.php" method="post">

  Имя: <input type="text" name="name" value="<h1>Иван</h1>"><p>

  Логин: <input type="text" name="login"
value="<script>alert('master')</script>"><p>

  E-mail: <input type="text" name="email"
```

```
value="john@mail.ru"><p>

Мой сайт: <input type="text" name="site" value="mysuper-
site.ru"><p>

Пароль: <input type="text" name="pwd" value="12345"><p>

<input type="submit" value="Отправить">

</form>
```

Перед отправкой данные формы приобрели следующий вид.

Программирование на языке PHP

php-course/form.html

Безопасность данных, часть 1

Валидация

Имя: <h1>Иван</h1>

Логин: <script>alert('master')</script>

E-mail: ♥ john@mail.ru

Мой сайт: 😘 mysuper-site.ru

Пароль: 12345

Отправить

Регулярные выражения не оценили старания пользователя и заблокировали прием данных текстовых полей.

example_2. server.php. Валидация регулярными выражениями

```
<?php

if(isset($_POST["name"])){

    // здесь будем собирать возможные ошибки валидации

    $_ERROR_VALID = array();

    // данные шаблоны регулярных выражений придумывались не
    для того чтобы запутать, а из соображений - чтобы попроче
```

```
// валидация поля Имя

if (!preg_match('/^[a-яёА-ЯЁ -]{3,10}$/u',
$_POST["name"])) $_ERROR_VALID[] = 'Имя';

// валидация поля Логин

if (!preg_match('/^[a-zA-Z0-9]{5,10}$/u',
$_POST["login"])) $_ERROR_VALID[] = 'Логин';

// валидация поля E-mail

if (!preg_match('/@/', $_POST["email"])) $_ERROR_VALID[]
= 'E-mail';

// валидация поля Мой сайт

if (!preg_match('/^[0-9a-za-я-]+\. [a-za-я]{2,5}$/iu',
$_POST["site"])) $_ERROR_VALID[] = 'Мой сайт';

// валидация поля Пароль

if (!preg_match('/^[0-9a-zA-Z!@#%$^&*]{6,15}$/u',
$_POST["pwd"])) $_ERROR_VALID[] = 'Пароль';

// по итогам выводим либо данные пользователя, либо
сообщение о возникших ошибках

if (count($_ERROR_VALID)) {

    echo "<h3>Неверно заполненные поля формы:</h3>";

    // выводим список полей не прошедших валидацию

    foreach ($_ERROR_VALID as $val) {

        printf ('Ошибка заполнения поля:<b>%s.</b><br />',
        $val);

    }

} else {

    echo "<h2>Добрый день, {$_POST['name']}!</h2>";

    echo "<h3>Мы получили от вас следующие
данные:</h3>";

    echo "Логин: {$_POST['login']} <p>";
```



```
echo "E-mail: {$_POST['email']} <p>";

echo "Ваш сайт: {$_POST['site']} <p>";

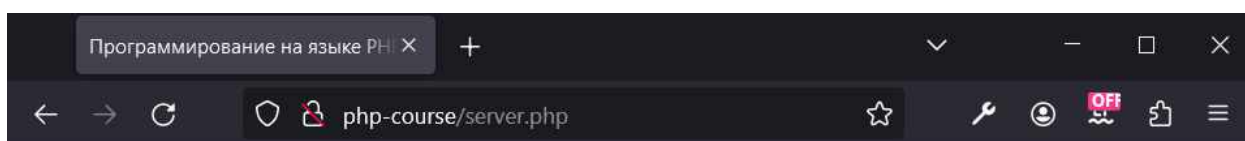
echo "Пароль: {$_POST['pwd']} <p>";

echo "<h3>Спасибо за проявленный интерес!</h3>";

}

}

?>
```



Безопасность данных, часть 1

Валидация

Неверно заполненные поля формы:

Ошибка заполнения поля: **Имя.**
Ошибка заполнения поля: **Логин.**
Ошибка заполнения поля: **Мой сайт.**
Ошибка заполнения поля: **Пароль.**

Это **валидация** – проверка данных на соответствие. Теперь выполним очистку данных – **санитизацию**.

Санитизация входных данных строковыми функциями

Чтобы свести к минимуму проблемы связанные с безопасной обработкой принимаемых данных рекомендуется использовать **специальные функции** очистки входных данных.

Примечание. Строковые функции мы рассматривали в уроке:

Встроенные функции, часть 2 курса.

trim

trim() — удаляет пробелы из начала и конца строки.

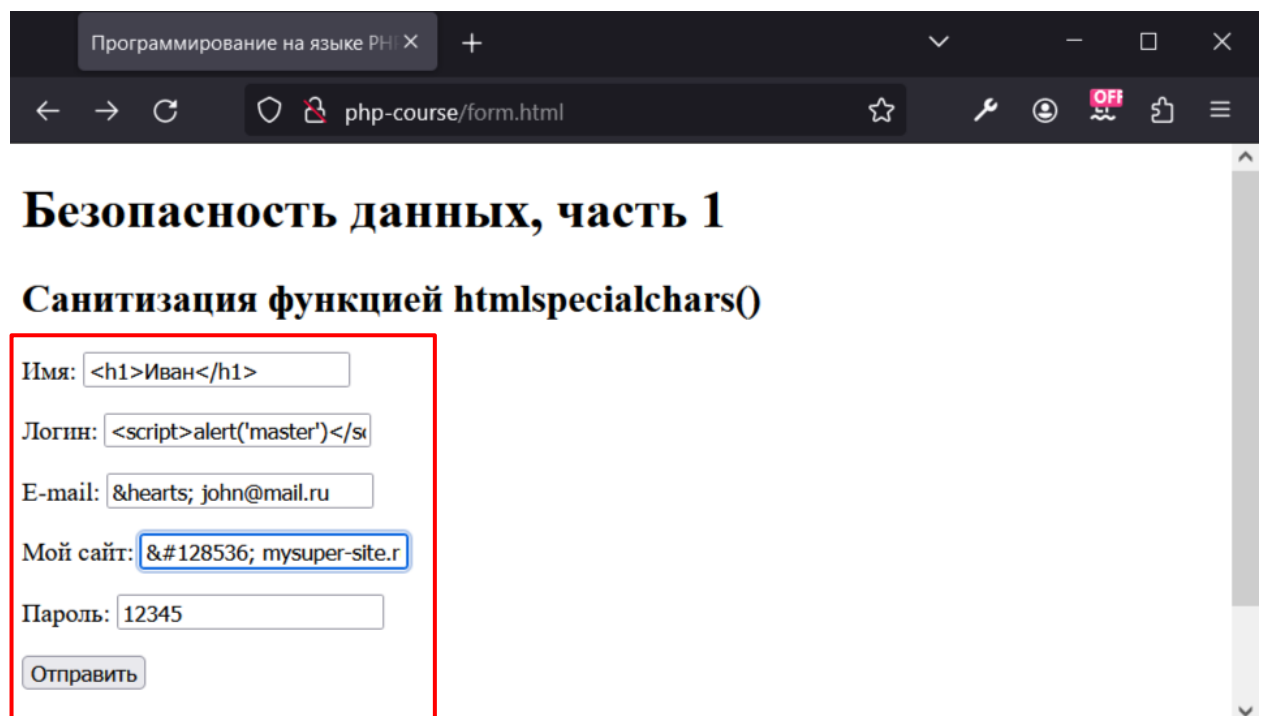
```
trim($string, $characters);
```

htmlspecialchars

htmlspecialchars() — преобразует некоторые специальные символы в HTML-сущности.

```
htmlspecialchars($string);
```

Файл формы и содержимое текстовых полей оставим без изменений.



Программирование на языке PHP X +

← → ↻ php-course/form.html ☆ 🔧 🛡️ OFF 📄 ☰

Безопасность данных, часть 1

Санитизация функцией htmlspecialchars()

Имя:

Логин:

E-mail:

Мой сайт:

Пароль:

example_3. server.php. Санитизация функцией htmlspecialchars()

```
<?php
```

```
if(isset($_POST["name"])){

    // очистим принятые данные

    $name = $_POST["name"] ? htmlspecialchars($_POST["name"])
    : "No name";

    $login = $_POST["login"] ?
    htmlspecialchars($_POST["login"]) : "нет данных";

    $email = $_POST["email"] ?
    htmlspecialchars($_POST["email"]) : "нет данных";

    $site = $_POST["site"] ? htmlspecialchars($_POST["site"])
    : "нет данных";

    $pwd = $_POST["pwd"] ? htmlspecialchars($_POST["pwd"]) :
    "нет данных";

    echo "<h2>Добрый день, $name!</h2>";

    echo "<h3>Мы получили от вас следующие данные:</h3>";

    echo "Логин: $login <p>";

    echo "Е-mail: $email <p>";

    echo "Ваш сайт: $site <p>";

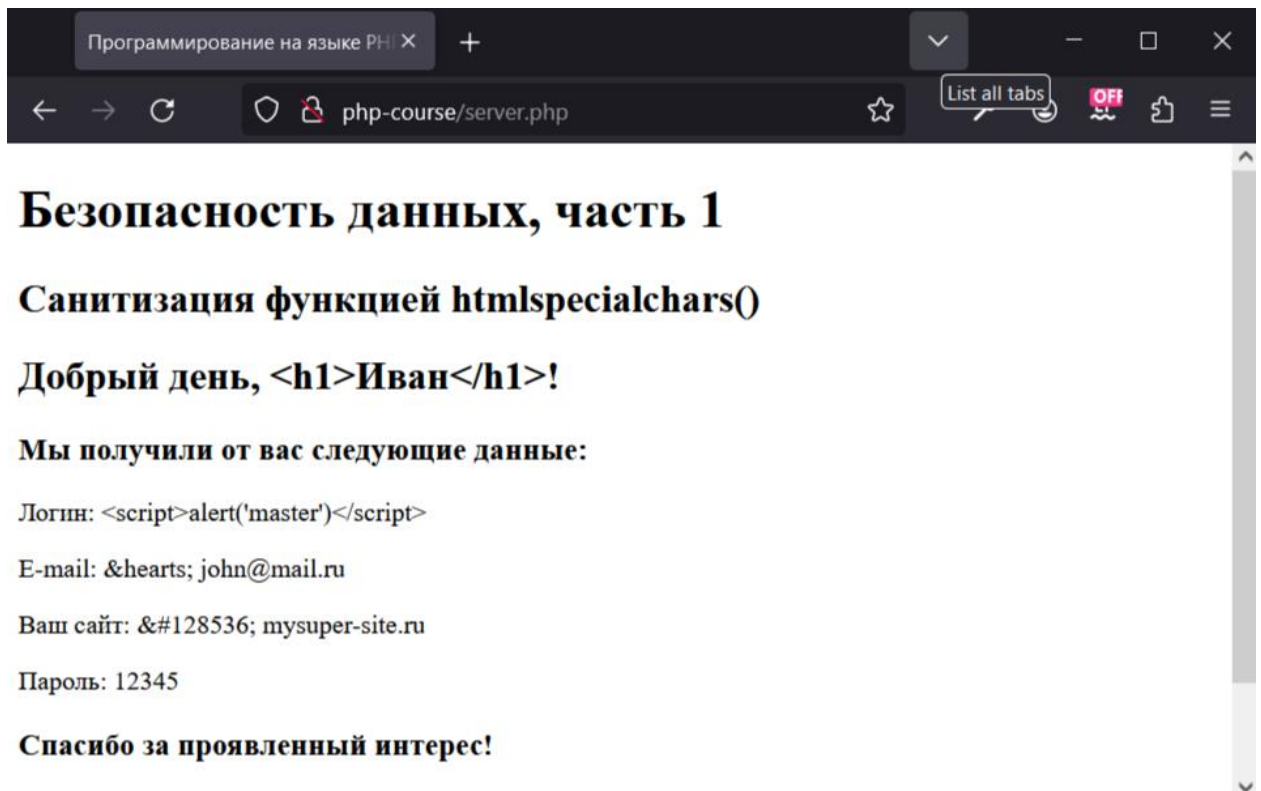
    echo "Пароль: $pwd <p>";

    echo "<h3>Спасибо за проявленный интерес!</h3>";

}
```

?>

Результат санитизации функцией **htmlspecialchars()** представлен на скриншоте ниже.



Очищенный от лишних символов вывод, выглядит грязновато, но результат достигнут.

Для сравнения используйте вывод без функции **htmlspecialchars()**. В файле **example_3/server-raw.php** предложен сценарий, выполняющий сырой (raw) вывод пользовательских данных.

Для тестирования не забудьте поменять метод **action** формы.

Примечание. RAW (raw — сырой, необработанный) — формат представления, содержащий необработанные (или обработанные в минимальной степени) данные.

example_3. server-raw.php

```
<?php

if(isset($_POST["name"])){

    echo "<h2>Добрый день, {$_POST["name"]}!</h2>";
```

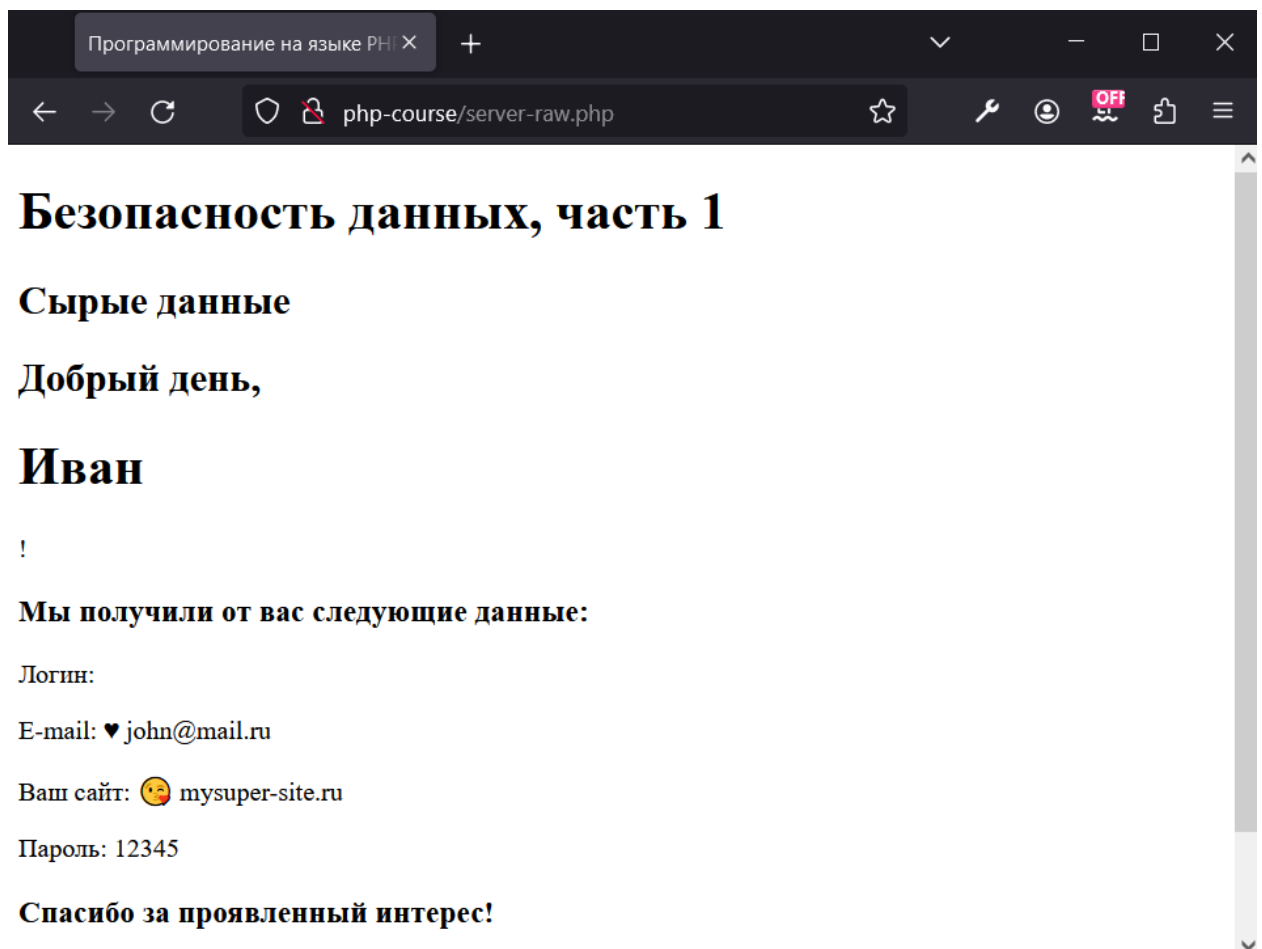
```
echo "<h3>Мы получили от вас следующие данные:</h3>";

echo "Логин: {"$_POST["login"]} <p>";
echo "E-mail: {"$_POST["email"]} <p>";
echo "Ваш сайт: {"$_POST["site"]} <p>";
echo "Пароль: {"$_POST["pwd"]} <p>";
echo "<h3>Спасибо за проявленный интерес!</h3>";

}
```

?>

Вывод *сырых* пользовательских данных представлен на следующем скриншоте.



htmlentities

htmlspecialchars() — преобразует все возможные символы в соответствующие **HTML-сущности**.

```
htmlspecialchars($string);
```

Файл формы и содержимое текстовых полей оставим без изменений.

example_4. server.php. Санитизация функцией htmlspecialchars ()

```
<?php

    if(isset($_POST["name"])){

        // очистим принятые данные

        $name = $_POST["name"] ? htmlspecialchars($_POST["name"]) :
        "No name";

        $login = $_POST["login"] ? htmlspecialchars($_POST["login"])
        : "нет данных";

        $email = $_POST["email"] ? htmlspecialchars($_POST["email"])
        : "нет данных";

        $site = $_POST["site"] ? htmlspecialchars($_POST["site"]) :
        "нет данных";

        $pwd = $_POST["pwd"] ? htmlspecialchars($_POST["pwd"]) : "нет
        данных";

        echo "<h2>Добрый день, $name!</h2>";

        echo "<h3>Мы получили от вас следующие данные:</h3>";

        echo "Логин: $login<p>";

        echo "Е-mail: $email<p>";

        echo "Ваш сайт: $site<p>";

        echo "Пароль: $pwd<p>";

        echo "<h3>Спасибо за проявленный интерес!</h3>";

    }

?>
```

Обе функции используются для вывода внешних данных, чтобы обезопасить веб-страницу от атак с использованием межсайтовых сценариев (XSS атак).

Разница между функциями **htmlspecialchars()** и **htmlspecialchars_decode()** в том, что тогда как **htmlspecialchars()** преобразует **все применимые символы в HTML-объекты**, **htmlspecialchars_decode()** преобразует только некоторые из них.

Таким образом **htmlspecialchars()** является подмножеством **htmlspecialchars_decode()**.

Важно. Если сомневаетесь какую функцию использовать, используйте **htmlspecialchars()**, - это лучшая функция для санитизации данных, отправляемых клиентом.

strip_tags

strip_tags() — удаляет теги HTML и PHP из строки.

```
strip_tags($string [, $allowed_tags]);
```

Файл формы и содержимое текстовых полей оставим без изменений.

example_5. server.php. Санитизация функцией strip_tags ()

```
<?php

if(isset($_POST["name"])){

    // очистим принятые данные

    $name = $_POST["name"] ? strip_tags($_POST["name"]) : "No
    name";

    $login = $_POST["login"] ? strip_tags($_POST["login"]) :
    "нет данных";

    $email = $_POST["email"] ? strip_tags($_POST["email"]) :
```

```
"нет данных";

$site = $_POST["site"] ? strip_tags($_POST["site"]) :
"нет данных";

$pwd = $_POST["pwd"] ? strip_tags($_POST["pwd"]) : "нет
данных";

echo "<h2>Добрый день, $name!</h2>";

echo "<h3>Мы получили от вас следующие данные:</h3>";

echo "Логин: $login <p>";

echo "Е-mail: $email <p>";

echo "Ваш сайт: $site <p>";

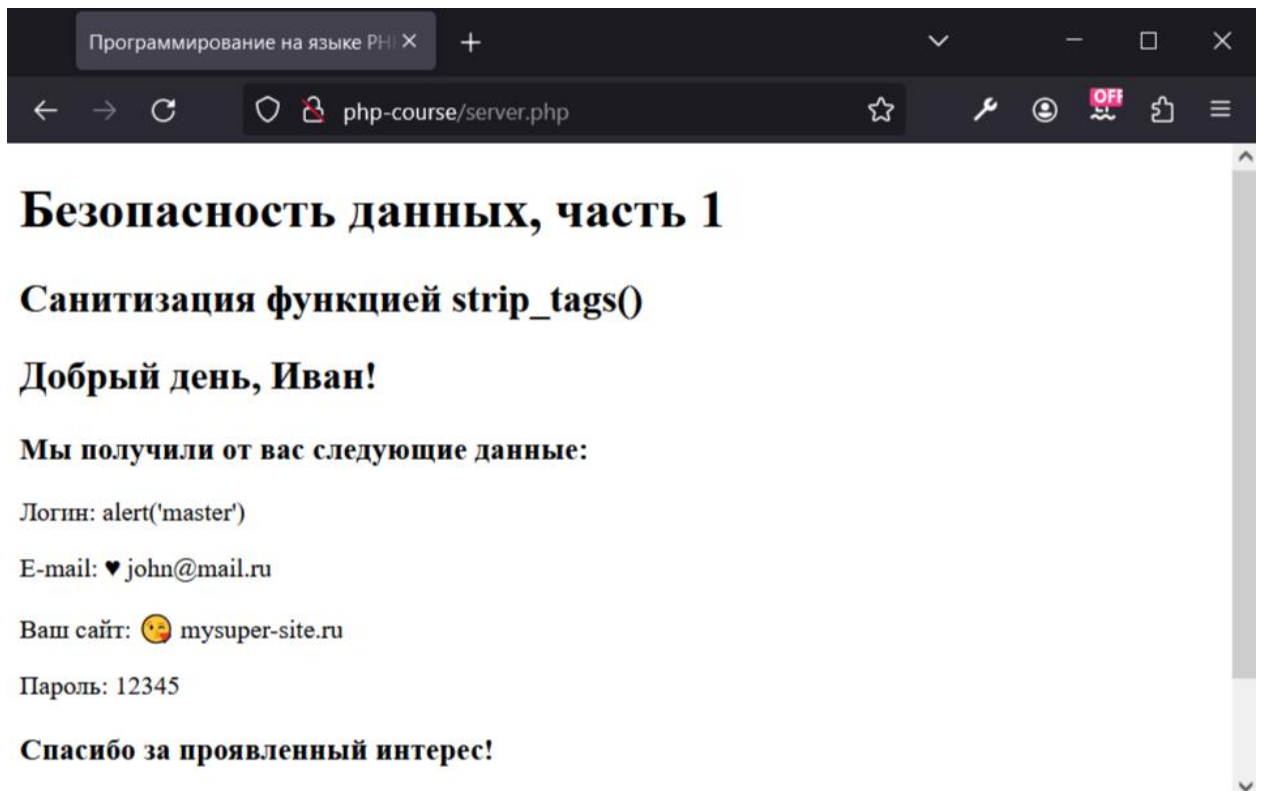
echo "Пароль: $pwd <p>";

echo "<h3>Спасибо за проявленный интерес!</h3>";

}

?>
```

Проанализируйте работу функции. Теперь удалены теги, но остались HTML-сущности.



Нет одной универсальной функции, есть набор функций, поставленная задача и голова чтобы думать.

Несколько вопросов, которые могут возникнуть в ходе выполнения урока.

1. **htmlspecialchars()** или **htmlspecialchars()** мы определили. Теперь, а **что лучше htmlspecialchars()** или **strip_tags()**?

Это зависит от того, важны ли HTML теги в полученных данных. Например, если мы имеем дело только с **именами** авторов и **названиями** статей, то, вероятно, можно безопасно использовать **strip_tags()**. Но если мы берем контент статьи целиком, он может содержать некоторые необходимые для отображения HTML теги, тогда, скорее всего **htmlspecialchars()**.

Важно. Все зависит от контекста выполняемой задачи.

2. **Зачем делать санитизацию, если есть валидатор?**

Ответов может быть несколько:

- Регулярное выражение вы пишете сами, санитизацию выполняет стандартная функция PHP, процент появления ошибки минимален
- Регулярное выражение может быть сложным, и не факт, что оно не будет включать в шаблон опасные символы.
- Некоторые поля вообще не поддаются описанию шаблона регулярного выражения.
- Береженого Бог бережет (русская пословица).

3. Что выполнять в первую очередь, санитизацию или валидацию?

Зависит от алгоритма работы вашего приложения. Может быть, важнее думать не о том, что делать первым, а о том, **к чему определенные действия могут привести** (например, код листинга example_9).

В следующих листингах приведу несколько разных подходов к обработке **невалидных** данных. Это, всего лишь, маленький пример разных алгоритмов.

В демонстрационном примере **example_6** при обнаружении невалидных данных прекращаем их обработку, выводим соответствующее сообщение.

example_6. form.html. Файл формы

```
<form action="server.php" method="post">
```

```
Имя: <input type="text" name="name" value="Иван"><p>
```

```
<input type="submit" value="Отправить">
```

```
</form>
```

example_6. server.php. Прекращаем обработку и выводим сообщение

```
<?php

    if(isset($_POST["name"])){

        // если поле невалидно:

        // - прекращаем обработку данных

        // - выводим сообщение с предложением попробовать еще раз

        // валидация поля Имя

        if (!preg_match('/^[a-яёА-ЯЁ -]{3,10}$/u',
$_POST["name"])) {

            echo "<h3>Неверно заполнено поле Имя</h3>";

            echo "Вернитесь назад и попробуйте еще раз: ";

            echo "<a href='javascript:
history.back()'>Назад</a>";

        } else {

            // какой-то контент сайта

            echo "<p>Лучшее время, чтобы посадить дерево, было 20
лет назад. Другое лучшее время - сейчас. | Китайская
пословица</p>";

        };

    }

?>
```

Обратите внимание, сырые данные из формы мы нигде не выводим, и тем более, никуда не сохраняем.

Демонстрационный код **example_7** немного сложнее. Пользовательская форма может быть открыта двумя способами:

- **прямая** ссылка;
- результат **перенаправления** из файла **server.php** в случае ошибки валидации данных.

При обнаружении невалидных данных прекращаем их обработку и перенаправляем на форму.

example_7. form.php. Файл формы

```
<?php

// если оказались в этом файле после переадресации
// выведем причину

if (isset($_GET["msg"])) {

    printf("

        <div style='margin-bottom:20px; padding:10px;
        border:1px solid green; color:green; font-
        style:italic'>%s</div>

        ",

        $_GET['msg']

    );

}

?>

<form action="server.php" method="post">

    Имя: <input type="text" name="name"
    value="<h1>Иван</h1>"><p>

    <input type="submit">

</form>
```

example_7. server.php. Прекращаем обработку и перенаправляем на форму

```
<?php

if (isset($_POST["name"])){

    // валидация поля Имя

    if (!preg_match('/^[a-яёА-ЯЁ -]{3,10}$/u',
```

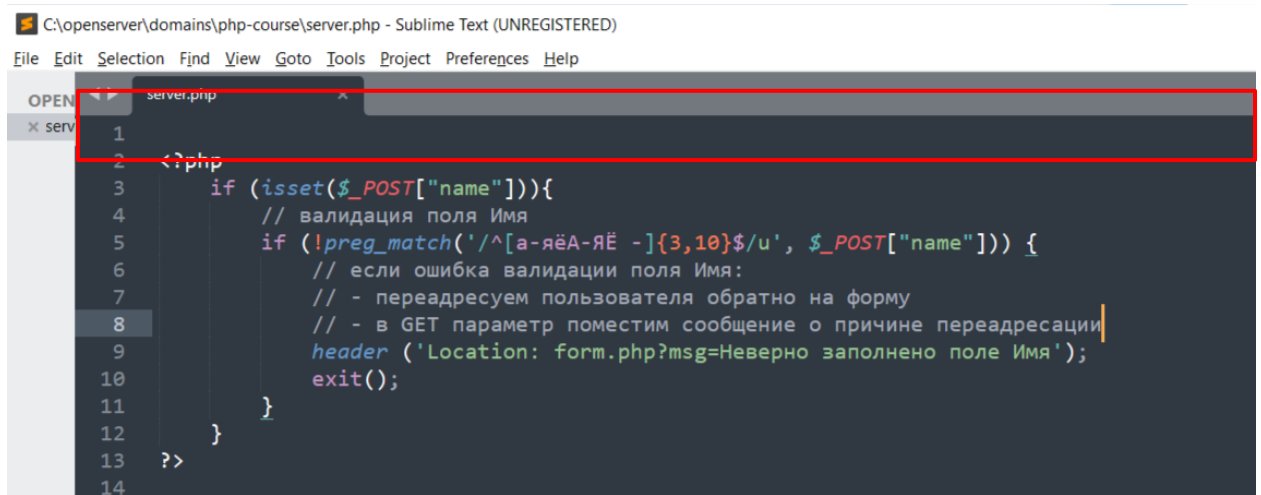
```
$_POST["name"])) {  
  
    // если ошибка валидации поля Имя:  
  
    // - переадресуем пользователя обратно на форму  
  
    // - в GET параметр поместим сообщение о причине  
    переадресации  
  
    header ('Location: form.php?msg=Неверно заполнено  
поле Имя');  
  
    exit();  
  
}  
  
}  
  
?>
```

Данные не выводим, в случае ошибки сразу **перенаправляем** пользователя на файл формы с выдачей причины перенаправления.

Важно. Функция **header()** подробно рассматривается в теме, посвященной взаимодействию PHP и MySQL.

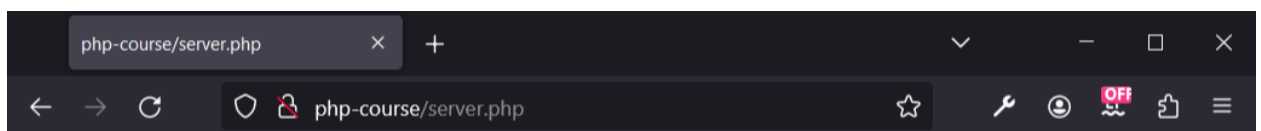
При использовании функции **header()** следите за тем, чтобы до использования функции не было **никаких выводов в браузер**, в том числе **пустых строк** и **пробелов**.

Так, даже простой перевод строки до открывающего тега `<?php` приведет к ошибке отправки заголовков: **Cannot modify header information – headers already sent by ...**



```
C:\openserver\domains\php-course\server.php - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

server.php
1
2 <?php
3     if (isset($_POST["name"])){
4         // валидация поля Имя
5         if (!preg_match('/^[a-яёА-ЯЁ -]{3,10}$/', $_POST["name"])) {
6             // если ошибка валидации поля Имя:
7             // - переадресуем пользователя обратно на форму
8             // - в GET параметр поместим сообщение о причине переадресации
9             header('Location: form.php?msg=Неверно заполнено поле Имя');
10            exit();
11        }
12    }
13    ?>
14
```



Warning: Cannot modify header information - headers already sent by (output started at C:\OpenServer\domains\php-course\server.php:1) in C:\OpenServer\domains\php-course\server.php on line 9

И еще один пример. При обнаружении невалидных данных продолжаем обработку данных, но предупреждаем пользователя.

example_8. form.html. Файл формы

```
<form action="server.php" method="post">

    Имя: <input type="text" name="name" value="Иван"><p>

    <input type="submit" value="Отправить">

</form>
```

example_8. server.php. Продолжаем обработку, но предупреждаем

```
<?php

if(isset($_POST["name"])) {

    // валидация поля Имя
```

```

if (!preg_match('/^[a-яёА-ЯЁ -]{3,10}$/u',
$_POST["name"])) {

    $name_valid = strip_tags($_POST["name"]);

    $msg = "<br />(поле 'Имя' невалидно, пожалуйста <a
href='javascript: history.back() '>исправьте</a>, или
программа заменит имя на '$name_valid')";

};

// очистим принятые данные

$name = htmlentities($_POST["name"]);

// несмотря на невалидность продолжаем работать с данными

echo "Добрый день: $name";

echo isset($msg)? $msg : '';

}

?>

```

Вот такие простые, но между тем разные подходы к обработке данных.

Еще раз. Важно представлять, в **какой момент** данные могут попасть в ваш скрипт и какие **последствия** от этого могут быть. Санитизация и валидация не соперничают за первенство, они дополняют друг друга.

А есть еще один показательный пример, который стоит особняком, но о нем тоже стоит сказать. О нем **обязательно** стоит сказать.

В демонстрационном примере **example_9** санитизацией меняем логин пользователя. Хорошо ли это...

example_9. form.html. Файл формы

```

<form action="server.php" method="post">

    Логин: <input type="text" name="login"

```

```
value="<h1>master2001</h1>"><p>

<input type="submit" value="Отправить">

</form>
```

example_9. server.php.

```
<?php

    if(isset($_POST["login"])){

        // то, что ввел пользователь

        $user_login = $_POST['login'];

        // то, что запомнит приложение (если пройдет валидацию)

        $sanitize_login = trim(strip_tags($_POST['login']));

        // проверяем данные на валидность

        if (!preg_match('/^[a-z0-9]{5,10}$/iu', $sanitize_login))
        {

            echo "<h3>Логин `{$sanitize_login}` невалиден</h3>";

        }

        echo "<h2>Что произошло:</h2>";

        echo "Пользователь ввел логин: <b>" .
            htmlspecialchars($user_login) . "</b> (не важно по какой
            причине, может мания у него ;)<p>";

        echo "Приложение запомнило (а может и записало в базу)
            логин: <b>" . $sanitize_login . "</b><p>";

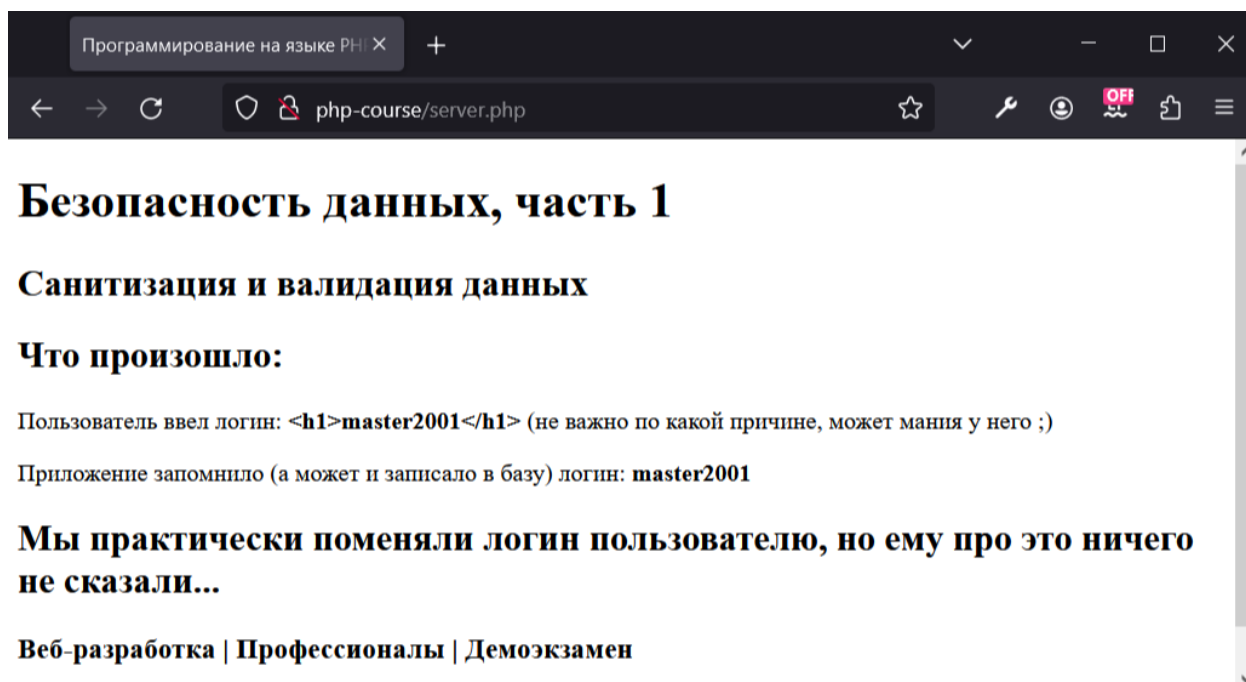
        echo "<h2>Мы практически поменяли логин пользователю, но
            ему про это ничего не сказали...</h2>";

    }

?>
```

Проанализируйте код. Подумайте о том, что произошло. Всегда думайте, что происходит в тех или иных инструкциях вашего кода (и все равно

ошибки будут, ошибки неизбежно сопровождают процесс программирования).



Если вы **не** (**умеете**|**хотите**|**доверяете**) себе писать регулярные выражения, вопрос и в этом случае может быть решен (частично).

Идем дальше.

Фильтрация входных данных с помощью фильтра

Начиная с PHP 5.xx производить очистку и валидацию данных стало проще с введением фильтрации данных **модулем PHP Filter**.

Фильтры модуля позволяют выполнить валидацию данных и обезопасить их от возможных вредоносных конструкций. Это особенно полезно, если содержимое получено из неизвестных или ненадежных источников, таких, как пользовательский ввод.

Например, ненадежные данные можно получить из **HTML-форм**.

filter_var

filter_var() — фильтрует переменную с помощью определённого фильтра.

Описание

```
filter_var($value, $filter = FILTER_DEFAULT, $options = 0);
```

Параметры

- **value** – значение переменной для фильтрации. Обратите внимание, что **скалярные** значения перед фильтрацией преобразуются к **строкам**.
- **filter** – идентификатор (ID) применяемого фильтра. Если не указан, то используется FILTER_DEFAULT, который обозначает, что по умолчанию не применяется никакого фильтра. Ниже приведены некоторые наиболее применяемые фильтры.
- **options** – ассоциативный массив параметров либо логическая дизъюнкция (операция ИЛИ) **флагов**. Флагов достаточно много, в примерах я представлю общий подход к применению.

Возвращаемые значения

Возвращает **отфильтрованные данные** или **false**, если фильтрация завершилась неудачей.

Типы фильтров

Фильтры валидации данных.

Идентификатор	Описание
FILTER_VALIDATE_EMAIL	Проверяет, что значение является корректным e-mail.
FILTER_VALIDATE_URL	Проверяет значение как URL. Помните, что URL не содержащий имя протокола http:// является корректным, так что может потребоваться дополнительная проверка того, что URL использует

	требуемый протокол, например ssh:// или mailto:. Функция считает корректными только URL, состоящие из символов ASCII. Интернациональные доменные имена не пройдут проверку.
FILTER_VALIDATE_DOMAIN	Проверяет, корректны ли длины меток имён домена. Проверяет доменные имена на соответствие стандартам. Опциональный флаг <code>FILTER_FLAG_HOSTNAME</code> добавляет возможность специально проверять имена хостов (они должны начинаться с букв, либо цифр и содержать только буквы, цифры и тире).
FILTER_VALIDATE_INT	Проверяет, что значение является корректным целым числом, и, при необходимости, входит в определённый диапазон, в случае успешной проверки преобразует в целое число. Перед сравнением строковые значения обрезаются с помощью функции <code>trim()</code>.
FILTER_VALIDATE_FLOAT	Проверяет, что значение является корректным числом с плавающей точкой, и, при необходимости, входит в определённый диапазон, в случае успешной проверки преобразует в число с плавающей точкой. Перед сравнением строковые значения обрезаются с помощью функции <code>trim()</code>.
FILTER_VALIDATE_BOOLEAN, FILTER_VALIDATE_BOOL	Возвращает <code>true</code> для значений "1", "true", "on" и "yes". Иначе возвращает <code>false</code>. Если установлен флаг <code>FILTER_NULL_ON_FAILURE</code>, то <code>false</code> возвращается только для значений "0", "false", "off", "no" и "", а <code>null</code> будет возвращён для всех значений не логического типа. Перед сравнением строковые значения обрезаются с помощью функции <code>trim()</code>.

* - полный набор констант и дополнительных флагов настроек можно посмотреть в открытых источниках

Очищающие фильтры.

Идентификатор	Описание
FILTER_SANITIZE_STRING	Удаляет теги и кодирует двойные и одинарные кавычки, при необходимости удаляет или кодирует специальные символы. Кодирование кавычек можно отключить, установив <code>FILTER_FLAG_NO_ENCODE_QUOTES</code> . (Объявлен устаревшим, начиная с PHP 8.1.0, используйте вместо него функцию <code>htmlspecialchars()</code>).
FILTER_SANITIZE_EMAIL	Удаляет все символы, кроме букв, цифр и <code>!#\$%&'*+,-=?^_`{ }~@.[]</code> .
FILTER_SANITIZE_URL	Удаляет все символы, кроме букв, цифр и <code>\$-_.+!*'(),{\} \^\~[]`<>#%"/?:@&=</code> .
FILTER_SANITIZE_ENCODED	Кодирует строку в формат URL, при необходимости удаляет или кодирует специальные символы.
FILTER_SANITIZE_NUMBER_INT	Удаляет все символы, кроме цифр и знаков плюса и минуса.
FILTER_SANITIZE_NUMBER_FLOAT	Удаляет все символы, кроме цифр, <code>+-</code> и, при необходимости, <code>.,eE</code> .

* - полный набор констант и дополнительных флагов настроек можно посмотреть в открытых источниках

Простой пример санитизации с использованием фильтра представлен в листинге кода **example_10**. Сделали и пошли дальше.

example_10. form.html. Файл формы

```
<form action="server.php" method="post">

    Мой сайт: <input type="text" name="site"
value="<script>alert('Hi!')</script><h1>http://mysuper-
site.ru</h1>"><p>

    <input type="submit" value="Отправить">

</form>
```

example_10. server.php. Простой пример санитизации фильтром

```
<?php

    if (isset($_POST['site'])) {

        // выполняем очистку данных

        echo filter_var($_POST['site'], FILTER_SANITIZE_URL);

    } else {

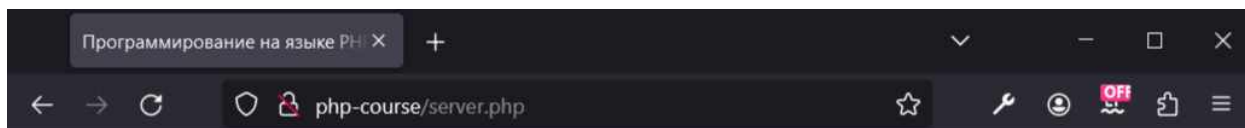
        echo "<h3>Введите данные в поле формы</h3>";

    }

?>
```

Стоп!

Вы выполнили код из листинга **example_10**. Вы очистили поле "Мой сайт" фильтром **FILTER_SANITIZE_URL**. Но, что на самом деле произошло? На выходе внедренный **JavaScript** и теги **h1**.



Безопасность данных, часть 1

Санитизация фильтром

http://mysuper-site.ru

Посмотрите описание идентификатора **FILTER_SANITIZE_URL** еще раз. Все правильно, все **символы текстового поля входят в число разрешенных**.

Замените идентификатор очищающего фильтра на **FILTER_SANITIZE_STRING** и сравните результат.

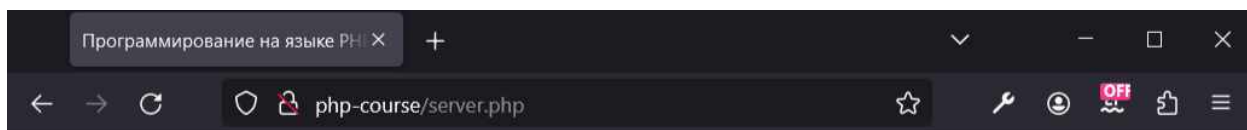
Примечание. Идентификатор фильтра **FILTER_SANITIZE_STRING** начиная с версии PHP 8.1 объявлен **устаревшим**.

Вместо **FILTER_SANITIZE_STRING** рекомендуется использовать функцию **htmlspecialchars()**. Другой способ – использовать функцию **error_reporting()**.

С помощью функции **error_reporting()** разработчики настраивают уровень **отчётности об ошибках** в соответствии с требованиями проекта.

```
// выключить протоколирование ошибок  
error_reporting(0);
```

Пример санитизации фильтром **FILTER_SANITIZE_STRING**.



Безопасность данных, часть 1

Санитизация фильтром

```
alert('Hi!')http://mysuper-site.ru
```

Примечание. Надеюсь, вы не собираетесь чистить URL идентификатором санитизации строки? Это разные фильтры, каждый хорош в своей области.

Рассмотрим пример санитизации и валидации фильтром поля **email**.

example_11. form.html. Файл формы

```
<form action="server.php" method="post">

    E-mail: <input type="text" name="email"
value="john@mail.ru.com.<h1>"><p>

    <input type="submit" value="Отправить">

</form>
```

example_11. server.php. Пример санитизации и валидации фильтром

```
<?php

    if (isset($_POST['email'])) {

        // очищаем данные

        $email = filter_var($_POST['email'],
FILTER_SANITIZE_EMAIL);

        echo "Результат санитизации: <b>" . $email . "</b><p>";

        // проверяем данные на валидность

        if (filter_var($email, FILTER_VALIDATE_EMAIL)) {

            echo "$email - <b>Валидный адрес</b>";

        } else {

            echo "$email - <b>НЕ валидный адрес</b>";

        }

    }

?>
```

Если вы оставили данные текстового поля, внесенные мной по умолчанию, то адрес **email** будет невалиден. И дело совсем не в доменном имени. В имени **john@mail.ru.com.h1** - нет символа "@".

Пример работы фильтра валидации с **дополнительным флагом** представлен в примере **example_12**.

В коде используются два флага, применение которых разрешено с фильтром **FILTER_VALIDATE_URL**:

- **FILTER_FLAG_PATH_REQUIRED** - требует, чтобы URL содержал часть с путём.
- **FILTER_FLAG_QUERY_REQUIRED** - требует, чтобы URL содержал часть со строкой запроса.

example_12. index.php. Пример валидации с дополнительным флагом

```
<?php

// примеры валидации с дополнительными флагами

echo "<pre>";

// флаг, требующий путь в строке адреса

// результат - false

var_dump(filter_var('https://vk.com', FILTER_VALIDATE_URL,
FILTER_FLAG_PATH_REQUIRED));

// результат - входная строка

var_dump(filter_var('https://vk.com/pechora_pro',
FILTER_VALIDATE_URL, FILTER_FLAG_PATH_REQUIRED));

echo "<p>-----</p>";

// флаг, требующий строки запроса

// результат - false

var_dump(filter_var('https://vk.com/pechora_pro',
FILTER_VALIDATE_URL, FILTER_FLAG_QUERY_REQUIRED));

// результат - входная строка

var_dump(filter_var('https://vk.com/pechora_pro?id=1',
FILTER_VALIDATE_URL, FILTER_FLAG_QUERY_REQUIRED));

?>
```


Практика фильтрации данных форм

Валидация входных данных форм

Как бы ни хороша была функция `filter_var()`, валидацию таких полей как пароль сделать не получится.

Примечание. Точнее сказать не так, сделать проверку немного сложнее, и все равно придется задействовать **регулярные выражения в синтаксисе функции**.

В следующем примере, какие-то данные проверим сами, какие-то отдадим на проверку функции фильтра.

example_13. form.html. Файл формы

```
<form action="server.php" method="post">

    Имя: <input type="text" name="name" value="Иван"><p>

    Логин: <input type="text" name="login" value="mastEr"><p>

    Е-mail: <input type="text" name="email"
value="jonn@mail.ru"><p>

    Мой сайт: <input type="text" name="site"
value="http://mysuper-site.ru"><p>

    Пароль: <input type="text" name="pwd" value="12345@%^&!"><p>

    <input type="submit" value="Отправить">

</form>
```

example_13. server.php. Валидация данных формы

```
<?php

    if(isset($_POST["login"])){
```

```
$_ERROR_VALID = array();

// ! используем регулярные выражения
// валидация поля Имя
if (!preg_match('/^[a-яёА-ЯЁ -]{3,10}$/u',
$_POST["name"])) $_ERROR_VALID[] = 'Имя';

// валидация поля Логин
if (!preg_match('/^[a-zA-Z0-9]{5,10}$/u',
$_POST["login"])) $_ERROR_VALID[] = 'Логин';

// валидация поля Пароль
if (!preg_match('/^[0-9a-zA-Z!@#$$%^&*]{6,15}$/u',
$_POST["pwd"])) $_ERROR_VALID[] = 'Пароль';

// ! используем фильтры
// валидация поля E-mail
if (!filter_var($_POST["email"], FILTER_VALIDATE_EMAIL))
$_ERROR_VALID[] = 'E-mail';

// валидация поля Мой сайт
if (!filter_var($_POST["site"], FILTER_VALIDATE_URL))
$_ERROR_VALID[] = 'Мой сайт';

// по итогам выводим либо данные пользователя, либо
сообщение об ошибке
if (count($_ERROR_VALID)) {

    echo "<h3>Неверно заполненные поля формы:</h3>";

    // выводим список полей не прошедших валидацию
    foreach ($_ERROR_VALID as $val) {

        printf ('Ошибка заполнения поля:
        <b>%s.</b><br>', $val);

    }

} else {

    echo "<h2>Добрый день, {$_POST['name']}!</h2>";

}
```

```

        echo "<h3>Мы получили от вас следующие
        данные:</h3>";

        echo "Логин: <b>{$_POST['login']}</b><br>";

        echo "E-mail: <b>{$_POST['email']}</b><br>";

        echo "Ваш сайт: <b>{$_POST['site']}</b><br>";

        echo "Пароль: <b>{$_POST['pwd']}</b><br>";

        echo "<h3>Спасибо за проявленный интерес!</h3>";

    }

}

?>

```

Санитизация входных данных форм

В последнем примере выполним санитизацию средствами фильтра. И опять вопросы к полю пароля.

Примечание. А нужно ли вообще очищать пароль?

- Чем сложнее и нечистоплотнее пароль, тем он безопаснее.
- Пароль все равно нужно хэшировать.

Но задача текущей темы не много думать, а больше проверять и чистить.

example_14. form.html. Файл формы

```
<form action="server.php" method="post">
```

```
Имя: <input type="text" name="name" value="<h1>Иван</h1>"><p>
```

```
Логин: <input type="text" name="login"
```

```
value="<script>alert('master')</script>"><p>
```

```
E-mail: <input type="text" name="email"
```

```
value="jonn@mail.ru.<com>"><p>

Мой сайт: <input type="text" name="site" value="mysuper-
site.ru"><p>

Пароль: <input type="text" name="pwd" value="12345@%^&!"><p>

<input type="submit" value="Отправить">

</form>
```

example_14. server.php. Санитизация данных формы

```
<?php

    // с помощью фильтра очистим принятые данные

    // очистка строки

    $name = $_POST["name"] ? filter_var($_POST["name"],
    FILTER_SANITIZE_STRING) : "No name";

    // очистка строки

    $login = $_POST["login"] ? filter_var($_POST["login"],
    FILTER_SANITIZE_STRING) : "нет данных";

    // очистка email

    $email = $_POST["email"] ? filter_var($_POST["email"],
    FILTER_SANITIZE_EMAIL) : "нет данных";

    // очистка url

    $site = $_POST["site"] ? filter_var($_POST["site"],
    FILTER_SANITIZE_URL) : "нет данных";

    // очистка строки

    $pwd = $_POST["pwd"] ? filter_var($_POST["pwd"],
    FILTER_SANITIZE_STRING) : "нет данных";

    echo "<h2>Добрый день, $name!</h2>";

    echo "<h3>Мы получили от вас следующие данные:</h3>";

    echo "Логин: $login <p>";
```

```
echo "E-mail: $email <p>";  
  
echo "Ваш сайт: $site <p>";  
  
echo "Пароль: $pwd <p>";  
  
echo "<h3>Спасибо за проявленный интерес!</h3>";
```

?>